

レポート課題 8

05-191009 奥村真司

問 1

考察

手で解いた結果はそれぞれ

```
[(alpha := Int),(beta := Int -> Int)]
TypeError
[(alpha := Int),(beta := Int)]
[(a1 := b1 -> b2),(a2 := b1 -> b2),(a3 := b1 -> b2)]
TypeError
```

で、これは上の問 4 で実装し、動作例で示した unify の結果と合致する。

問 2

動作例

```
# #use "tySyntax.ml";;
val ty_subst : (tyvar * ty) list -> ty -> ty = <fun>
....
# let sigma = [(1,TyInt);(2,TyFun(TyInt,TyInt))] in
  let t = TyFun((TyVar 1),(TyVar 2)) in
    ty_subst sigma t;;
- : ty = TyFun (TyInt, TyFun (TyInt, TyInt))
```

考察

TyInt, TyBool ならばそのまま返し、TyFun(x,y) ならば x,y を再帰的に substitute する。TyVar id ならば sigma を id で検索し、なかったらそのまま、あれば置き換える、というように実装した。

問 3

動作例

```
# let sigma1 = [(1,TyInt)] in
  let sigma2 = [(2,(TyVar 1))] in
    let sigma1_sigma2 = compose sigma1 sigma2 in
```

```
ty_subst sigma1 sigma2 (TyVar 2);;
- : ty = TyInt
```

考察

はじめ σ_2 を σ_1 より前に concat するだけにしていたが、上の動作例のように多段で変換される時に正しく動かなかったので関数合成にちゃんとなるように修正した。まず、`domain : subst -> tyvar list` という σ の定義域のような (正確には定義域ではないが、`subst` の `list` のペアの左側に現れる `tyvar` の集合、これ以外の `tyvar` は置き換えられずそのままになる) ものを求める関数を定義した。compose では σ_1, σ_2 の `domain` の和集合を求め、その元 `tv` に対し `ty_subst sigma1 (ty_subst sigma2 (TyVar tv))` を割り当てるような `subst` を作って返すようにした。

問 4

動作例

```
# #use "tySyntax.ml";;
.....
# let alpha = TyVar( new_tyvar () ) in
  let beta = TyVar ( new_tyvar () ) in
  [(alpha, TyInt); (beta, TyFun(alpha, alpha))];;
- : (ty * ty) list = [(TyVar 1, TyInt); (TyVar 2, TyFun (TyVar 1, TyVar 1))]
# let alpha = TyVar( new_tyvar () ) in
  let beta = TyVar ( new_tyvar () ) in
  unify [(alpha, TyInt); (beta, TyFun(alpha, alpha))];;
- : (tyvar * ty) list = [(3, TyInt); (4, TyFun (TyInt, TyInt))]
# let alpha = TyVar( new_tyvar () ) in
  unify [(TyInt, TyFun(TyInt, alpha))];;
Exception: TyError.
# let alpha = TyVar( new_tyvar () ) in
  let beta = TyVar ( new_tyvar () ) in
  unify [(TyFun(TyInt, TyInt), TyFun(alpha, beta))];;
- : (tyvar * ty) list = [(6, TyInt); (7, TyInt)]
# let a1 = TyVar (new_tyvar ()) in
  let a2 = TyVar (new_tyvar ()) in
  let a3 = TyVar (new_tyvar ()) in
  let b1 = TyVar (new_tyvar ()) in
  let b2 = TyVar (new_tyvar ()) in
  unify [(TyFun(a1, a2), TyFun(a2, a3)); (a3, TyFun(b1, b2))];;
- : (tyvar * ty) list =
[(8, TyFun (TyVar 11, TyVar 12)); (9, TyFun (TyVar 11, TyVar 12));
 (10, TyFun (TyVar 11, TyVar 12))]
# let alpha = TyVar (new_tyvar ()) in
  let beta = TyVar (new_tyvar ()) in
  let gamma = TyVar (new_tyvar ()) in
  unify [(TyFun(alpha, alpha), TyFun(beta, gamma)); (gamma, TyFun(TyInt, beta))];;
Exception: TyError.
```

考察

動作例は問 1 の各制約集合に対して最汎単一化子を求めたものである。問 1 の考察でも述べたとおりこの結果は手で計算した結果と合致する。まず出現検査のために型と型変数を受け取ってその型に型変数が含まれているかを bool 値で返す関数 `include_tyvar` を定義した。これは `ty_subst` と同じように型の構造に関して帰納的に求めればよい。あとは講義の単一化アルゴリズムのスライドを参考にしながらパターンマッチで条件分岐して求めた。一度バグらせた原因として、制約集合から選んだ 1 つの制約が両方型変数でかつそれらが一致する時は、スライドの 2 つめのケース ($\{s = s\} \cup C$ のケース) に相当し、4,5 番目のケースでないということがあった。スライドの順番通りに判定しないと思わぬ落とし穴があった。

動作例

```
# let f = fun x -> fun y -> x;;
for all a2 a1
  ( a1 -> ( a2 -> a1 ))
val f = <fun>
# let rec fix f = fun x -> f (fix f) x;;
for all a6 a3
  ( ( ( a3 -> a6 )-> ( a3 -> a6 ))-> ( a3 -> a6 ))
val fix = <rec fun>
# let rec f n = if n = 0 then 0 else n + f (n-1) in f 10;;
for all
  int
- = 55
# let apply = fun f -> fun x -> let g = f in if true then f 1 else f true;;
Fatal error: exception TySyntax.TyError
-----
# let id = fun x -> x in
  let apply = fun f -> if f true then f 1 else f false in
  apply id;;
Fatal error: exception TySyntax.TyError
-----
# let id = fun x -> x in
  if id true then id 1 else id 3;;
for all
  int
- = 1
# let id = fun x -> x in
  if id true then id 1 else id false;;
Fatal error: exception TySyntax.TyError
```

考察

スライドの方針にしたがって実装した。多数の補助関数を作成した。スライドの内容を理解するのに時間がかかった部分としては、「型環境に現れる」というのは量子子に束縛されていないもので、型環境の変数の型スキーマの中に現れるという意味であるということだった

た。量子子で束縛されたものに関しては名前が重要なのではないので変数の「自由な出現」と束縛されたものの区別を意識しながら実装した。instantiate は型スキーマを受け取って量子子に束縛された変数を全て新たな型変数で置き換えた型を返した。get_type_vars は型に含まれる型変数のリストを求めるもので、ty_subst と同様に型の構造に関して帰納的に求めれば良い。この他に、型スキーマを受け取って量子子に束縛されていない型変数のリストを返す get_free_tyvars_from_tysc や型環境の全ての変数の型スキーマの、自由に出現する変数の集合の和集合を求める get_free_tyvars_from_tyenv, σ での置き換えを引数のリスト内の型変数にのみ行う ty_subst_limited などを実装した。generalize は、型推論が終わった後に未決定のまま残っている型変数のうち、型環境で自由に出現しない型変数たちに量子子をつけて型スキーマに拡張する関数であり、定義した補助関数たちを用いて実装した。eval.ml の infer_expr は型環境の定義の変更に伴って、適宜変更し、let in 等型環境を変更するものについてもスライドを参考にしながら変更を施した。実装しているときに let in 式の制約収集のスライドで行われている $\Delta = \Gamma\sigma$ の変換が必要なのか疑問に思った。なぜなら型環境が更新される際は未決定の型変数全てが一般化されて量子子に束縛されるのでこの変換で型環境は変化しないのではないかと思ったからである。友人たちに相談してみるとこの置き換えをしないとうまく行かない例として `fun x -> let g = fun y -> x y in g 4` というものを教えてもらえた。関数抽象の評価の途中では型環境の中に量子子に束縛されていない型変数を含む状態 (この例では x) で評価されることがあるので Δ の必要性がわかった。

発展 1

考察

まず今まで実装してきた型システムにおける型は、

- `int, bool` は型
- `s, t` が型の時 `s->t` は型

というように帰納的に構成されており、情報論理の term のように深さ有限の木構造で表せるものを考えている。 $\alpha = t(t \text{ は } \alpha \text{ でない})$ という形の制約を満たすものは、`t` 中の α に `t` を代入することを繰り返すことでいくらでも `t` の構成を表す木の深さがいくらでも大きくなる。なので出現検査を行わなければならない。出現検査をしないことは、再帰的な型を考えることに相当する。

発展 2

動作例

```
# (1,2);;
for all
  ( int,  int)
- = (1,2)
# (1,1);;
for all
  (same pair  int)
- = (1,same)
```

```

# let x = fun x -> (x,x) in
  x (fun x -> x);;
for all a3
(same pair ( a3 -> a3 ))
- = (<fun>,same)
# let x = fun x -> (x,x) in
  let x = fun y -> x (x y) in
    x (fun x -> x);;
for all a12
(same pair (same pair ( a12 -> a12 )))
- = (((<fun>,same),same),same)
# let x = fun x -> (x,x) in
  let x = fun y -> x (x y) in
    let x = fun y -> x (x y) in
      let x = fun y -> x (x y) in
        let x = fun y -> x (x y) in
          x (fun x -> x);;
for all a41
(same pair (same pair (same pair (same pair (same pair (same pair
(same pair (same pair (same pair (same pair (same pair (same pair
(same pair (same pair (same pair (same pair (same pair (same pair
(same pair (same pair (same pair (same pair (same pair (same pair
(same pair (same pair (same pair (same pair (same pair (same pair
(same pair (same pair ( a41 -> a41 ))))))))))))))))))))))))
- = (((((((((((((((((((((((((((((((((((((((((((((((((((((((<fun>,same),same),same),same),
same),same),same),same),same),same),same),same),same),same),same),same),same),same),
same),same),same),same),same),same),same),same),same),same),same),same),same),
same),same),same),same),same),same)

```

考察

いままでのインタプリタで上の式の型推論の結果が返ってこないのは、infer_expr の計算量が非常に大きいからである。式の評価結果の値の型は非常に大きいのでなんども再帰的に infer_expr が呼び出される。ここで infer_expr の引数に着目すると、何度も同じ引数で呼ばれることがわかる。何度も同じ引数で再計算するのは無駄なので、フィボナッチ数を求める場合などと同様にメモ化したくなった。しかし関数型言語でメモ化するのは副作用をあつかうことになり第4回の課題でやったようにモナドを用いなければならない。それは面倒なので、今回の式では同じものの組が何度も作られることに着目し、TyPair(x,y) の型推論をするときに、x と y が同一ならば一方しか評価しないようにした。(メモ化と比較すると多少一般性は失われてしまうが)

次に型の表し方についての工夫を説明する。実際の値の型を真面目に扱うのは難しそうなのでより短い形式で表すことを考える。上の式では同じもののペア (x,x) が何度も作られるのでこれを (x,same) と表すことにする。内部的にも TySamePair というものを用意した。こうすることで、例えば (((x,x),(x,x)),((x,x),(x,x))) ならば (((x,same),same),same) と短く書ける。このように型推論において、同じもののペアを特別に扱うことは式の値ではなく式 (型 expr) から判断するので、高速に判定できる。(値 (value) は表現すると非常に大きいかもしれないが、構文 (expr) は上の7行を parse したもので短い。) これで今回の式の値のようなものの巨大な型を短い形式で表すことができた。以上の2つの工夫によって動作例のように与えられた式の型推論ができるようになった。

発展 3

要約

unification アルゴリズムの停止性を示すために、structural recursion でない unification アルゴリズムを structural recursion の形で書けるように再定式化している。structural recursion で書くことができれば停止性は定義より明らかとなるので、そのように定式化することで停止性が証明される。これ以降出てくる帰納的な定義はすべて構造帰納のものである。

2 章では“ n 種類の型の集合の元”の型である $\text{Fin } n$ の帰納的定義が与えられている。直感的には以下の表のようになる。また、structural recursion で書ける関数のクラスは primitive recursive

$\text{Fin } 1$	$f0_0$
$\text{Fin } 2$	$f0_1 \quad fs_0 f0_0$
$\text{Fin } 3$	$f0_2 \quad fs_1 f0_1 \quad fs_2 fs_1 f0_0$

function のクラスよりも広いことが述べられている (アッカーマン関数とその例として挙げられている。) $\text{Fin } n$ のような Inductive family を用いることで general recursion 内の再帰構造を明示的な structural recursion に変えることができる。

3 章では $\text{Fin } n$ の値 ($\text{Fin } n$ は型の型なのでこれは型であることに注意) のインスタンスなどの型 $\text{Term } n$ が帰納的に定義されている。

4 章では occurrence check について thin,thick という関数を通して説明されている。thin は、 $\text{Fin } n$ の値の集合から、 $\text{Fin } sn$ の値の集合から 1 つを除いた集合への全単射を定めており、帰納的に定義されている。 $\text{thin}_n x y$ はその全単射での $\text{Fin } n$ の値 y の行き先で、 $\text{Fin } sn$ の値の集合から x を除いた集合の元である。thin の逆関数のようなものが thick で、 x と y が同じ時は no (OCaml のオプション型の None に相当) を返し、そうでない場合は $\text{thick } x$ は $\text{thin } x$ の逆関数のようになっている。この thick も帰納的に定義されている。 $\text{Fin } sn$ の値 x と $\text{Term } sn$ の値 t について、 t に x が現れるかどうかは thick を用いて帰納的に検査できる。

5 章では $m, n \in \mathbb{N}$ で添字付けられた型 $\text{AList } m \ n$ の値を syntax として substitution (これは $\text{Fin } m \rightarrow \text{Term } n$ のような型をもつ) の semantics が帰納的に定義されている。また、 $\text{AList } m \ n$ の n は重要ではなく、ある n が存在することが重要なので n と型 $\text{AList } m \ n$ の組を新たな型 $\exists(\text{AList } m)$ の値としている。

6 章ではここまでで準備した構造帰納で定義されたものを用いて unification アルゴリズムを構造帰納で書いている。特に着目すべきは amgu の 8 つめのケースでサイズが 1 つ落ちていることである。flexFlex, flexRigid の場合も thick や check を用いて書けるので全体として構造帰納になっている。はじめにも述べた通り、構造帰納で書けたことによりただちに unification アルゴリズムの停止性がわかる。

感想

$\text{Fin } n$ のように型の型 (Inductive family) を考えるという発想が初めてで面白いと思った。 $\text{Fin } n, \text{Term } n$ だけに限らず、 thin のような全単射の構成も構造帰納で綺麗に定義でき、unification アルゴリズムが最終的に様々な面で何重にも構造帰納を含む形で書けるのが美しいと思った。読んでいる途中で syntax と semantics を混同して混乱してしまったが、もう一度読み直すとしっかりとこれらが分けて与えられていてわかりやすかった。 dependent type (依存型) という単語が何度も現れたので調べてみたがよくわからなかったので理解したい。